



Universidad
Francisco de
Vitoria

UFV Madrid



MEMORIA DEL PILAR 1 — INGENIERÍA DEL DATO

CLASIFICACIÓN AUTOMÁTICA DE RESIDUOS RECICLABLES MEDIANTE DEEP LEARNING

Automatic Sorting of Recyclable Waste Using Deep Learning

Alumno: Pablo Huidobro García

Email: 9005411@alumnos.ufv.es

Grado en Business Analytics · Universidad Francisco de Vitoria

Curso Académico: 2025-2026

ÍNDICE

ÍNDICE DE TABLAS	3
ÍNDICE DE ILUSTRACIONES	4
1. Introducción y Objetivos del Pilar	5
1.1 Posición del Pilar en la Arquitectura del TFG	5
1.2 Objetivos Específicos del Pilar 1	6
2. Descripción del Dataset	7
2.1 Origen y Naturaleza de los Datos: Dataset Propio	7
2.2 Clases, Volumen Total y Distribución	7
2.3 Análisis de Desbalance y Decisión sobre Rebalanceo	8
2.4 Condiciones de Captura y Variabilidad Visual	8
2.5 Organización del Dataset en Disco	9
2.6 Justificación de la Arquitectura de Almacenamiento	10
3. Preprocesamiento y Preparación de Datos	11
3.1 Visión General del Pipeline	11
3.2 Resolución de Entrada: 224×224 Píxeles	11
3.3 Normalización: Estadísticas de ImageNet	12
3.4 Data Augmentation: Técnicas Seleccionadas y Justificación	13
3.4.1 Técnicas descartadas y su justificación	14
3.4.2 Orden de aplicación de RandomErasing	15
3.5 Pipeline de Transformaciones: Código Implementado	15
Transformaciones de entrenamiento (transform_train)	15
Transformaciones de validación y test (transform_val_test)	15
4. División del Dataset y Prevención de Data Leakage	16
4.1 Proporciones del Split y Tamaños Resultantes	16
4.2 Justificación de la Proporción 70/15/15	16
4.3 Reproducibilidad: Semilla Fija seed = 42	17
4.4 Prevención de Data Leakage: Tres Mecanismos Complementarios	17
4.5 Configuración de los DataLoaders	17
5. Scripts y Notebooks Utilizados	19
5.1 Notebook Principal: pilar1_ingenieria_dato.ipynb	19
5.2 Dependencias y Tecnologías	20
5.3 Ficheros de Salida Generados por el Pilar 1	21
6. Resultados, Visualizaciones y Métricas del Pilar 1	22
6.1 Resumen de Resultados del Análisis Exploratorio	22
6.2 Visualizaciones Generadas	23
6.2.1 Distribución del Dataset (distribucion_dataset.png)	23

6.2.2 Ejemplos del Dataset (ejemplos_dataset.png).....	24
6.2.3 Estadísticas RGB por canal y clase (estadisticas_rgb.png)	24
6.2.4 Distribución de resoluciones originales (resoluciones_originales.png)	25
6.2.5 Efecto del data augmentation (augmentation_efecto.png)	26
7. Conclusiones y Consideraciones para Producción	27
7.1 Síntesis del Pilar 1	27
7.2 Consideraciones para el Deployment en Producción (SDDR).....	28
. Consideraciones para el deployment del pipeline en producción (SDDR).....	28
7.3 Limitaciones Identificadas	29
8.Referencias Bibliográficas	31

ÍNDICE DE TABLAS

Tabla 1 Estructura del TFG por pilares: denominación, peso y estado	5
Tabla 2. Distribución de imágenes por clase en el dataset de residuos reciclables	7
Tabla 3. Condiciones de captura y dimensiones de variabilidad visual del dataset.....	9
Tabla 4. Estadísticas RGB del dataset propio comparadas con ImageNet-1K	12
Tabla 5. Técnicas de data augmentation: parámetros y justificación bibliográfica	13
Tabla 6. Distribución del dataset: conjuntos de entrenamiento, validación y test (70/15/15)	16
Tabla 7. Estructura del notebook pilar1_ingenieria_dato.ipynb: secciones y descripción	19
Tabla 8. Dependencias y tecnologías utilizadas en el Pilar 1	20
Tabla 9. Ficheros de salida generados por el Pilar 1: ruta, contenido y uso posterior	21
Tabla 10. Resumen de métricas y parámetros del análisis exploratorio del Pilar 1	22
Tabla 11. Consideraciones para el deployment del pipeline en producción (SDDR)	28

ÍNDICE DE ILUSTRACIONES

Ilustración 1. Distribución del dataset de residuos reciclables por clase.....24

Ilustración 2. Ejemplos del dataset por clase: cuadrícula 3×5 con 5 muestras por clase24

Ilustración 3. Estadísticas RGB del Dataset vs ImageNet25

Ilustración 4. Distribución de resoluciones originales26

Ilustración 5. Efecto del Data Augmentation27

1. Introducción y Objetivos del Pilar

El presente Pilar 1 constituye la base sobre la que se asienta todo el proyecto de clasificación automática de residuos reciclables. En el marco del Grado en Business Analytics de la Universidad Francisco de Vitoria (UFV), el Pilar de Ingeniería del Dato representa el 20 % de la calificación total del TFG y tiene por objeto garantizar que los datos empleados en el entrenamiento de los modelos de Deep Learning sean de alta calidad, estén correctamente documentados y permitan la reproducibilidad completa del experimento.

El proyecto global evalúa la viabilidad de redes neuronales convolucionales (CNN) modernas para la clasificación automática de envases de bebidas reciclables —latas de aluminio, botellas de plástico PET y botellas de vidrio— en condiciones representativas de los sistemas de depósito, devolución y retorno (SDDR). Este contexto adquiere especial relevancia por la Ley 7/2022, de 8 de abril, de residuos y suelos contaminados para una economía circular (BOE núm. 85), que establece la obligación de implantar un sistema SDDR en España antes de noviembre de 2026, así como por el Reglamento Europeo de Envases y Residuos de Envases (Parlamento Europeo y Consejo de la UE, 2024), que extiende esta obligación al conjunto de la UE con horizonte 2029.

1.1 Posición del Pilar en la Arquitectura del TFG

El TFG está organizado en tres pilares interdependientes que deben ejecutarse de forma secuencial:

Tabla 1 Estructura del TFG por pilares: denominación, peso y estado

Pilar	Denominación	Peso (%)	Estado
Pilar 1	Ingeniería del Dato	20 %	Completo
Pilar 2	Análisis de los Datos y Modelado	20 %	Completo
Pilar 3	Análisis de Negocio	20 %	Completo

Nota. Elaboración propia a partir del README.md del proyecto, sección «Estado del Proyecto».

La dependencia entre pilares es estricta y deliberada: el Pilar 1 genera el fichero `split_indices.json`, que contiene los índices exactos de las imágenes asignadas a cada partición (train/val/test). El Pilar 2 carga estos índices directamente, sin recalcularlos, lo que garantiza que el conjunto de test del Pilar 2 es idéntico al definido en el Pilar 1. Esta arquitectura de trazabilidad elimina cualquier posibilidad de data leakage entre pilares y asegura la reproducibilidad completa del experimento.

1.2 Objetivos Específicos del Pilar 1

De acuerdo con el anteproyecto aprobado (sección 2), los objetivos específicos del Pilar de Ingeniería del Dato son:

- Recopilar y organizar un dataset original de imágenes propias capturadas en condiciones deliberadamente heterogéneas que repliquen la variabilidad inherente a entornos SDDR operativos.
- Realizar un análisis exploratorio del dataset: distribución por clase, cuantificación del desbalance y verificación de la variabilidad visual.
- Diseñar e implementar el pipeline de preprocesamiento (redimensionado, normalización, data augmentation) con justificación bibliográfica de cada decisión de diseño.
- Generar una partición reproducible del dataset (train/val/test) mediante semilla fija (seed = 42), preservando los índices en `split_indices.json` para garantizar la trazabilidad con el Pilar 2.
- Documentar todas las decisiones metodológicas con rigor académico, citando las fuentes que las respaldan, de forma que el experimento sea completamente replicable.

2. Descripción del Dataset

2.1 Origen y Naturaleza de los Datos: Dataset Propio

El dataset empleado en este TFG es completamente original: todas las imágenes han sido capturadas directamente por el autor, sin recurrir a repositorios públicos como TrashNet, WasteNet u otros frecuentemente utilizados en la literatura de clasificación de residuos.

Esta decisión responde a uno de los objetivos nucleares del proyecto: abordar la brecha entre el rendimiento en laboratorio y el rendimiento en deployment real. Los trabajos de Zhang et al. (2023) y Du et al. (2024) demuestran que la degradación de precisión cuando un modelo entrenado en laboratorio se despliega en entornos operativos se atribuye fundamentalmente a la discrepancia entre las condiciones de entrenamiento y las condiciones reales: iluminación variable, fondos complejos y oclusiones. Los datasets públicos disponibles fueron capturados en condiciones de laboratorio controladas y con categorías de residuos que no se corresponden con el dominio específico de los sistemas SDDR españoles (tres materiales: aluminio, PET y vidrio).

Al capturar las imágenes en condiciones deliberadamente heterogéneas, el dataset refleja la variabilidad operativa real de un sistema SDDR, mejorando la validez ecológica del experimento y reduciendo la dependencia del data augmentation sintético como única fuente de diversidad.

Nota. Elaboración propia. Metadatos documentados en dataset_metadata.json (02_Ingenieria_de_Dato/). Justificación metodológica en el anteproyecto aprobado, secciones 2 y 5.1.

2.2 Clases, Volumen Total y Distribución

El dataset está compuesto por tres clases correspondientes a los envases de bebidas contemplados por el sistema SDDR en España:

Tabla 2. Distribución de imágenes por clase en el dataset de residuos reciclables

Clase	Descripción	Nº de imágenes	Porcentaje (%)
LATAS	Latas de aluminio de bebidas	743	32,8 %
PET	Botellas de plástico PET	764	33,7 %
VIDRIO	Botellas de vidrio	757	33,4 %
TOTAL	—	2.264	100,0 %

Nota. Datos extraídos de dataset_metadata.json, campos «clases» y «total_imagenes».

El tamaño total del dataset (2.264 imágenes) supera el umbral superior del rango previsto en el anteproyecto (900-1.500 imágenes), lo que proporciona una base más robusta para el entrenamiento. Este tamaño es coherente con trabajos recientes en clasificación de residuos con datasets propios (Malik et al., 2022; Olorunnisola et al., 2025) y permite demostrar la efectividad del transfer learning con datos limitados, que es precisamente uno de los objetivos del proyecto: cuantificar el impacto de transfer learning y data augmentation estratégico en la robustez de modelos entrenados con datasets de tamaño reducido.

2.3 Análisis de Desbalance y Decisión sobre Rebalanceo

El desbalance máximo entre clases, calculado como $(\max - \min) / \max \times 100$, es de únicamente un 2,7 % (diferencia de 21 imágenes entre PET con 764 y LATAS con 743). Este nivel es estadísticamente despreciable y sitúa el dataset en la categoría de balanceado según el criterio implementado en el notebook del Pilar 1 (umbral del 10 %).

En consecuencia, no se aplican técnicas de mitigación activa como weighted loss, oversampling sintético (SMOTE) o focal loss. Esta decisión está alineada con las recomendaciones de Sokolova y Lapalme (2009), quienes demuestran matemáticamente que, ante desbalances leves y costes de error similares entre clases, el F1-Score ponderado es la métrica adecuada y el entrenamiento con Cross-Entropy estándar no introduce sesgo apreciable. La elección del F1-Score ponderado como métrica principal en el Pilar 2 garantiza que el rendimiento se evalúa de forma equitativa entre las tres clases.

2.4 Condiciones de Captura y Variabilidad Visual

Para replicar las condiciones heterogéneas de un sistema SDDR real, las imágenes fueron capturadas bajo cuatro dimensiones de variabilidad sistemáticamente cubiertas:

Tabla 3. Condiciones de captura y dimensiones de variabilidad visual del dataset

Dimensión de variabilidad	Condiciones cubiertas
Iluminación	Natural diurna, artificial interior fría/cálida, iluminación mixta (natural + artificial)
Fondo	Uniforme, texturizado, complejo con elementos distractores
Ángulo de captura	Frontal, lateral, oblicuo, superior
Estado del envase	Nuevo (sin uso), usado, deteriorado/parcialmente deformado, con etiquetas desprendidas

Nota. Elaboración propia. Datos recogidos en dataset_metadata.json, campo «condiciones_captura»; anteproyecto, sección 5.1.

La cobertura sistemática de estas dimensiones está justificada por la evidencia empírica de Zhang et al. (2023) y Du et al. (2024), que demuestran que modelos con 95 % de precisión en datasets de laboratorio pueden caer hasta un 75-80 % en entornos operativos reales con iluminación variable y fondos complejos. Incorporar esta variabilidad desde el origen del dataset —en lugar de depender únicamente del augmentation sintético— reduce la brecha de generalización de forma más efectiva.

2.5 Organización del Dataset en Disco

Los datos brutos están organizados en una estructura de directorios plana, directamente compatible con la clase ImageFolder de torchvision, que infiere automáticamente las etiquetas de clase a partir del nombre de cada subdirectorio:

```
02_Ingenieria_de_Dato/
├── Datos brutos/
│   ├── LATAS/ ← 743 imágenes (.jpg, .jpeg, .png)
│   └── PET/ ← 764 imágenes (.jpg, .jpeg, .png)
```

```
| └── VIDRIO/ ← 757 imágenes (.jpg, .jpeg, .png)
|   ├── dataset_metadata.json
|   ├── split_indices.json
|   └── Resultados/
|       ├── distribucion_dataset.png
|       └── ejemplos_dataset.png
```

2.6 Justificación de la Arquitectura de Almacenamiento

La estructura de almacenamiento adoptada —directorios organizados por clase bajo `02_Ingenieria_de_Dato/Datos brutos/`— es el estándar de facto para datasets de imágenes en PyTorch, compatible nativamente con `torchvision.datasets.ImageFolder`. Esta elección maximiza la inspectabilidad directa del etiquetado, la portabilidad entre sistemas operativos y la compatibilidad con el pipeline de entrenamiento, sin el overhead de esquema de las bases de datos relacionales ni la pérdida de inspectabilidad de los formatos binarios empaquetados (HDF5). El fichero `dataset_metadata.json` complementa esta arquitectura documentando los metadatos del dataset de forma estructurada y legible por máquina.

3. Preprocesamiento y Preparación de Datos

3.1 Visión General del Pipeline

El pipeline de preprocesamiento está implementado mediante la API `transforms.Compose` de `torchvision` y se divide en dos variantes diferenciadas según el conjunto al que se aplica. Esta separación es una decisión metodológica fundamental: el `data augmentation` se aplica exclusivamente al conjunto de entrenamiento, de modo que la evaluación sobre validación y test se realiza siempre sobre imágenes en su estado original (solo redimensionadas y normalizadas), garantizando una estimación no sesgada del rendimiento real del modelo.

3.2 Resolución de Entrada: 224×224 Píxeles

Todas las imágenes se redimensionan a 224×224 píxeles mediante `transforms.Resize((224, 224))`. Esta resolución es la estándar de preentrenamiento de las tres arquitecturas evaluadas (ResNet50, EfficientNet-B3 y ConvNeXt-Tiny) en ImageNet-1K, conforme documentan sus autores respectivos (He et al., 2016; Tan y Le, 2019; Liu et al., 2022).

Esta elección se justifica por cuatro razones complementarias:

- **Compatibilidad con los pesos preentrenados:** los tres modelos aprenden sus representaciones con esta resolución. Utilizar una resolución diferente requeriría interpolación de los pesos de las primeras capas convolucionales, introduciendo inconsistencias y reduciendo la efectividad del transfer learning.
- **Estándar de facto en la literatura:** permite comparación directa con los resultados del estado del arte publicados por los autores de las arquitecturas.
- **Capacidad representacional suficiente:** 224×224 px proporciona suficiente detalle para discriminar los patrones visuales de los tres materiales (transparencia del vidrio, brillo metálico de las latas, textura plástica del PET) sin sacrificar eficiencia computacional.
- **Viabilidad en hardware SDDR:** resoluciones superiores (384×384 o 512×512) incrementarían el tiempo de inferencia de forma cuadrática —puesto que el coste computacional de las convoluciones escala con el área de la imagen— comprometiendo el throughput requerido por sistemas SDDR (1-5 envases por segundo).

Desde el punto de vista técnico, `transforms.Resize((224, 224))` aplica interpolación bilineal por defecto y redimensiona la imagen a un cuadrado exacto de 224×224 píxeles sin preservar la relación de aspecto original. Esto implica que imágenes con proporciones no cuadradas (p. ej., capturas verticales de botellas) experimentan una distorsión geométrica moderada. Esta distorsión no compromete el rendimiento del clasificador por dos razones: (1) las redes convolucionales profundas aprenden representaciones invariantes a deformaciones afines moderadas gracias a la profundidad de sus jerarquías de características (He et al., 2016); y (2) el dataset incluye suficiente variabilidad intrínseca (ángulos oblicuos, distintas proporciones de captura) para que el modelo no se sobreajuste a proporciones geométricas específicas. El análisis de resoluciones originales (sección 6.2.4) confirma el rango de proporciones presentes en el dataset.

Nota. pilar1_ingenieria_dato.ipynb, celda 8 (`CONFIG['image_size'] = 224`); He et al. (2016); Tan y Le (2019); Liu et al. (2022).

3.3 Normalización: Estadísticas de ImageNet

Tras la conversión a tensor (`ToTensor()`), se aplica normalización canal a canal con `media = [0.485, 0.456, 0.406]` y `desviación típica = [0.229, 0.224, 0.225]`, que corresponden a las estadísticas del dataset ImageNet-1K calculadas sobre 1,2 millones de imágenes de entrenamiento.

Podría plantearse calcular las estadísticas directamente sobre el dataset propio. Sin embargo, esta alternativa presenta dos inconvenientes fundamentales: (1) con solo 2.264 imágenes, las estadísticas resultantes serían menos representativas; y (2) los pesos preentrenados fueron aprendidos bajo la distribución de ImageNet, por lo que aplicar estadísticas distintas alteraría la distribución de activaciones en las primeras capas, reduciendo la efectividad del transfer learning.

No obstante, para validar empíricamente esta elección, se han calculado las estadísticas RGB del dataset propio por canal y por clase (véase Tabla 4 y figura `estadisticas_rgb.png`, sección 6.2.3). Los resultados confirman que las medias y desviaciones típicas del dataset propio son próximas a las de ImageNet, lo que valida la elección de normalización. Las diferencias observadas son atribuibles a la especificidad del dominio (envases sobre fondos urbanos) y no justifican un cambio de estadísticas que comprometería la compatibilidad con los pesos preentrenados.

Tabla 4. Estadísticas RGB del dataset propio comparadas con ImageNet-1K

Clase	μR	μG	μB	σR	σG	σB
LATAS	0,417	0,386	0,344	0,186	0,177	0,167
PET	0,501	0,471	0,434	0,194	0,186	0,179
VIDRIO	0,469	0,424	0,370	0,189	0,183	0,178
ImageNet	0,485	0,456	0,406	0,229	0,224	0,225

Nota. Valores calculados sobre el dataset completo (2.264 imágenes). Fuente: rgb_stats.json generado por la celda 1.4 del notebook

3.4 Data Augmentation: Técnicas Seleccionadas y Justificación

El data augmentation se aplica on-the-fly en cada época de entrenamiento: las transformaciones aleatorias se ejecutan en el momento en que el DataLoader carga cada imagen, generando una versión diferente en cada pasada. Esto implica que el modelo nunca ve exactamente la misma versión de una imagen dos veces, generando variabilidad genuina y efectiva sin incremento del almacenamiento en disco. Esta aproximación es metodológicamente superior a un dataset aumentado estático (que se había contemplado inicialmente con factor 5×), que simplemente replicaría imágenes con transformaciones fijas aplicadas una sola vez y no aportaría variabilidad real durante el entrenamiento.

Las cuatro técnicas seleccionadas y su justificación son las siguientes:

Tabla 5. Técnicas de data augmentation: parámetros y justificación bibliográfica

Técnica	Parámetros	Justificación operativa y bibliográfica
RandomHorizontalFlip	p = 0,50	Los envases son visualmente simétricos respecto al eje vertical. Un volteo horizontal produce una imagen semánticamente válida que simula orientaciones alternativas al depositar el envase en la máquina SDDR. Clasificada por Zeng (2024) como transformación geométrica básica con alto impacto en generalización.

Técnica	Parámetros	Justificación operativa y bibliográfica
RandomRotation	$\pm 15^\circ$	Simula ligeras inclinaciones del envase durante la deposición. Zeng (2024) recomienda rotaciones moderadas ($\leq 30^\circ$) para evitar artefactos semánticos: rotaciones superiores a 45° generarían posiciones físicamente imposibles en un SDDR. Se optó por $\pm 15^\circ$ como valor conservador dentro del rango seguro.
ColorJitter	brightness = 0,2; contrast = 0,2; saturation = 0,2; hue = 0,1	Simula variaciones de iluminación (cambios de brillo y contraste) y diferencias de color superficial (saturación, hue) documentadas en el protocolo de captura. Mahmood y Minallah (2023) confirman que las transformaciones fotométricas moderadas incrementan la robustez ante cambios de iluminación sin alterar semánticamente la clase.
RandomErasing	p = 0,30; scale = (0,02–0,15)	Simula oclusiones parciales del envase: manos del usuario, etiquetas desprendidas, suciedad superficial. Zeng (2024) lo identifica como técnica de regularización efectiva. El scale (2–15 % del área) es suficientemente pequeño para no ocultar las características discriminativas del material. Se aplica después de ToTensor() y Normalize() por requisito técnico de PyTorch.

Nota. pilar1_ingenieria_dato.ipynb, celda 24; Zeng (2024); Mahmood y Minallah (2023); anteproyecto, sección 5.2.

3.4.1 Técnicas descartadas y su justificación

Se descartaron Mixup y CutMix porque generan imágenes híbridas combinando píxeles de dos clases distintas, produciendo ejemplos semánticamente incoherentes en el dominio de la clasificación de residuos (p. ej., mezcla de lata y PET). Las rotaciones superiores a $\pm 30^\circ$ y los volteos verticales se descartaron por generar posiciones físicamente imposibles en un sistema SDDR. Los resultados del Pilar 2 (F1-Score > 98 %) confirman que el nivel de augmentation seleccionado es suficiente.

3.4.2 Orden de aplicación de RandomErasing

Es preciso destacar que RandomErasing se aplica después de ToTensor() y Normalize(), no antes. Esta es una restricción técnica de PyTorch: RandomErasing opera sobre tensores (no sobre imágenes PIL), sustituyendo regiones con valores de tensor (cero o ruido). Aplicarlo antes de ToTensor() generaría un error en tiempo de ejecución.

3.5 Pipeline de Transformaciones: Código Implementado

Transformaciones de entrenamiento (transform_train)

```
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
    transforms.RandomErasing(p=0.3, scale=(0.02, 0.15)), # después de ToTensor
])
```

Transformaciones de validación y test (transform_val_test)

```
transform_val_test = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
])
```

Nota. pilar1_ingenieria_dato.ipynb, celda 18.

4. División del Dataset y Prevención de Data Leakage

4.1 Proporciones del Split y Tamaños Resultantes

El dataset se divide en tres conjuntos no solapados con la proporción 70 % entrenamiento / 15 % validación / 15 % test, establecida explícitamente en el anteproyecto aprobado (sección 5.5):

Tabla 6. Distribución del dataset: conjuntos de entrenamiento, validación y test (70/15/15)

Conjunto	Proporción	Nº de imágenes	Función
Train	70 %	1.584	Entrenamiento de los modelos con data augmentation on-the-fly
Validation	15 %	339	Monitoreo del entrenamiento y activación de early stopping
Test	15 %	341	Evaluación final no sesgada (uso único al final del entrenamiento)
TOTAL	100 %	2.264	–

Nota. pilar1_ingenieria_dato.ipynb, celda 26; split_indices.json.

Los tamaños exactos se calculan como: $\text{train_size} = \text{int}(0,70 \times 2.264) = 1.584$; $\text{val_size} = \text{int}(0,15 \times 2.264) = 339$; $\text{test_size} = 2.264 - 1.584 - 339 = 341$.

4.2 Justificación de la Proporción 70/15/15

La proporción 70/15/15 ha sido elegida frente a la alternativa habitual 80/10/10 por razones estadísticas vinculadas al tamaño reducido del dataset. Con solo 2.264 imágenes totales:

- Un 15 % de validación (~339 imágenes) proporciona una estimación más estable de la pérdida de validación que un 10 % (~226 imágenes), reduciendo la varianza en la detección del punto óptimo de early stopping. Con muestras pequeñas, la pérdida de validación puede oscilar significativamente entre épocas, dificultando la identificación temprana de overfitting.
- Un 15 % de test (~341 imágenes, ~114 por clase) ofrece mayor representatividad estadística para las métricas finales que un 10 % (~226 imágenes, ~75 por clase). Un

conjunto de test más amplio reduce la varianza de las estimaciones de F1-Score y precision/recall por clase.

- La proporción 70/15/15 es coherente con la práctica recomendada en la literatura para datasets de tamaño limitado (< 3.000 imágenes), donde la representatividad de los conjuntos de evaluación es prioritaria sobre maximizar el conjunto de entrenamiento.

4.3 Reproducibilidad: Semilla Fija seed = 42

El split se genera mediante permutación aleatoria de los índices con semilla fija seed = 42, fijada simultáneamente en PyTorch (`torch.manual_seed(42)`), NumPy (`np.random.seed(42)`) y Python random (`random.seed(42)`). Esto garantiza que cualquier investigador puede re-ejecutar el notebook y obtener exactamente la misma partición. Los índices se persisten en `split_indices.json` y el Pilar 2 los carga directamente, eliminando por diseño cualquier riesgo de partición inconsistente entre pilares.

```
Indices=torch.randperm(n, generator=torch.Generator().manual_seed(42)).tolist()
```

4.4 Prevención de Data Leakage: Tres Mecanismos Complementarios

La implementación del Pilar 1 incorpora tres mecanismos independientes para garantizar la ausencia de data leakage:

- Tres instancias independientes de ImageFolder: se crean tres objetos dataset separados (`dataset_train`, `dataset_val`, `dataset_test`) con sus transformaciones respectivas. `dataset_train` recibe `transform_train` (con augmentation); `dataset_val` y `dataset_test` reciben `transform_val_test` (sin augmentation). Esto garantiza que el augmentation no contamina accidentalmente los conjuntos de evaluación.
- Split generado una única vez y persistido: el Pilar 1 calcula la partición, la guarda en `split_indices.json` y el Pilar 2 la carga directamente. No existe ninguna fase en la que los índices se recalculen, eliminando el riesgo de particiones inconsistentes.
- Evaluación final en test realizada una sola vez: el conjunto de test no se utiliza en ningún momento durante el desarrollo del Pilar 2 (ni para ajuste de hiperparámetros ni para selección de arquitectura). Solo se evalúa al final del entrenamiento completo, garantizando una estimación no sesgada del rendimiento real.

4.5 Configuración de los DataLoaders

Los tres DataLoaders se configuran con `batch_size = 32` y `num_workers = 2`. El conjunto de entrenamiento usa `shuffle = True` para evitar que el orden de presentación de las imágenes introduzca sesgos en el aprendizaje; los conjuntos de validación y test usan `shuffle = False` para garantizar evaluaciones deterministas y reproducibles.

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=2)
val_loader   = DataLoader(val_dataset,  batch_size=32, shuffle=False, num_workers=2)
test_loader  = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=2)
```

5. Scripts y Notebooks Utilizados

5.1 Notebook Principal: pilar1_ingenieria_dato.ipynb

Ruta: 02_Ingenieria_de_Dato/notebooks/pilar1_ingenieria_dato.ipynb

El notebook implementa el pipeline completo del Pilar 1 en una estructura ETL (Extracción, Transformación, Carga) claramente diferenciada, siguiendo las mejores prácticas de ingeniería del dato para experimentos reproducibles:

Tabla 7. Estructura del notebook pilar1_ingenieria_dato.ipynb: secciones y descripción

Sección	Celda(s)	Descripción
Configuración de rutas y entorno	2	Definición portable de rutas mediante <code>pathlib.Path</code> (compatible con Windows, macOS y Linux)
Instalación de dependencias	4	<code>timm</code> , <code>albumations</code> , <code>seaborn</code> (instalación programática para garantizar reproducibilidad)
Importación de librerías	6	<code>torch</code> , <code>torchvision</code> , <code>numpy</code> , <code>matplotlib</code> , <code>json</code> , <code>random</code> , <code>warnings</code>
Configuración global (CONFIG)	8	<code>seed = 42</code> , <code>image_size = 224</code> , <code>batch_size = 32</code> , <code>num_workers = 2</code> . Fijación de semillas en PyTorch, NumPy y random
1.1 EXTRACCIÓN: Análisis del Dataset	10	Conteo de imágenes por clase, cálculo del desbalance máximo, evaluación automática balanceado/desbalanceado
1.2 Visualización de la distribución	12	Gráfico de barras (nº de imágenes) y diagrama de sectores (porcentajes). Guardado en Resultados/
1.3 Visualización de ejemplos	14	Grid 3×5 con 5 muestras aleatorias por clase para verificación visual del etiquetado
1.4 Estadísticas RGB	16	Cálculo de media y std por canal R/G/B para cada clase. Comparación con ImageNet.

Sección	Celda(s)	Descripción
		Generación de estadísticas_rgb.png y rgb_stats.json
1.5 Resoluciones originales	18	Histogramas de anchos, altos y ratios de aspecto antes del resize. Generación de resoluciones_originales.png
1.6 Efecto augmentation	20	Grid visual 3×5 con original + 4 versiones aumentadas por clase. Generación de augmentation_efecto.png
1.7 Guardar metadata del dataset	22	Serialización de dataset_metadata.json con clases, totales, porcentajes, desbalance y condiciones de captura
1.8 TRANSFORMACIÓN: Preprocesamiento	24	Definición de transform_train (con augmentation) y transform_val_test (sin augmentation)
1.9 CARGA: DataLoaders	26	Split reproducible (seed = 42), tres ImageFolder independientes, DataLoaders, guardado de split_indices.json

Nota. pilar1_ingenieria_dato.ipynb, celdas 2-26.

5.2 Dependencias y Tecnologías

Tabla 8. Dependencias y tecnologías utilizadas en el Pilar 1

Librería / Herramienta	Uso en el Pilar 1
PyTorch ($\geq 2.1.0$)	DataLoader, Dataset, transforms, semillas de reproducibilidad (torch.manual_seed, torch.randperm)
torchvision ($\geq 0.16.0$)	ImageFolder, transforms.Compose y todas las transformaciones de preprocesamiento y augmentation
NumPy	Operaciones numéricas y semilla np.random.seed

Librería / Herramienta	Uso en el Pilar 1
Matplotlib	Visualizaciones de distribución por clase y ejemplos del dataset
timm	Instalada en el Pilar 1; utilizada en el Pilar 2 para cargar las arquitecturas CNN preentrenadas
json (built-in)	Serialización de dataset_metadata.json y split_indices.json
pathlib (built-in)	Gestión portable de rutas de ficheros (compatible con sistemas operativos distintos)

5.3 Ficheros de Salida Generados por el Pilar 1

Tabla 9. Ficheros de salida generados por el Pilar 1: ruta, contenido y uso posterior

Fichero	Ruta	Contenido y uso posterior
dataset_metadata.json	02_Ingenieria_de_Dato/	Metadatos del dataset: clases, totales, porcentajes, desbalance, condiciones de captura. Referencia documental permanente.
split_indices.json	02_Ingenieria_de_Dato/	Índices de las imágenes en cada partición (train/val/test). Consumido directamente por el Pilar 2 para garantizar el mismo split.
distribucion_dataset.png	02_Ingenieria_de_Dato/Resultados/	Gráfico de barras y diagrama de sectores de la distribución del dataset por clase.
ejemplos_dataset.png	02_Ingenieria_de_Dato/Resultados/	Grid 3×5 con 5 ejemplos visuales por clase para verificación del etiquetado.

Fichero	Ruta	Contenido y uso posterior
rgb_stats.json	02_Ingenieria_de_Dato/	Estadísticas RGB por canal y clase. Valida la normalización ImageNet.
estadisticas_rgb.png	Resultados/	Gráfico comparativo de medias y desviaciones RGB del dataset vs. ImageNet.
resoluciones_originales.png	Resultados/	Histogramas de resoluciones nativas. Contextualiza el resize a 224×224 px.
augmentation_efecto.png	Resultados/	Grid de ejemplos antes/después del augmentation por clase.

6. Resultados, Visualizaciones y Métricas del Pilar 1

6.1 Resumen de Resultados del Análisis Exploratorio

Tabla 10. Resumen de métricas y parámetros del análisis exploratorio del Pilar 1

Métrica / Parámetro	Valor
Total de imágenes en el dataset	2.264
Número de clases	3 (LATAS, PET, VIDRIO)
Imágenes por clase — LATAS	743 (32,8 %)
Imágenes por clase — PET	764 (33,7 %)
Imágenes por clase — VIDRIO	757 (33,4 %)

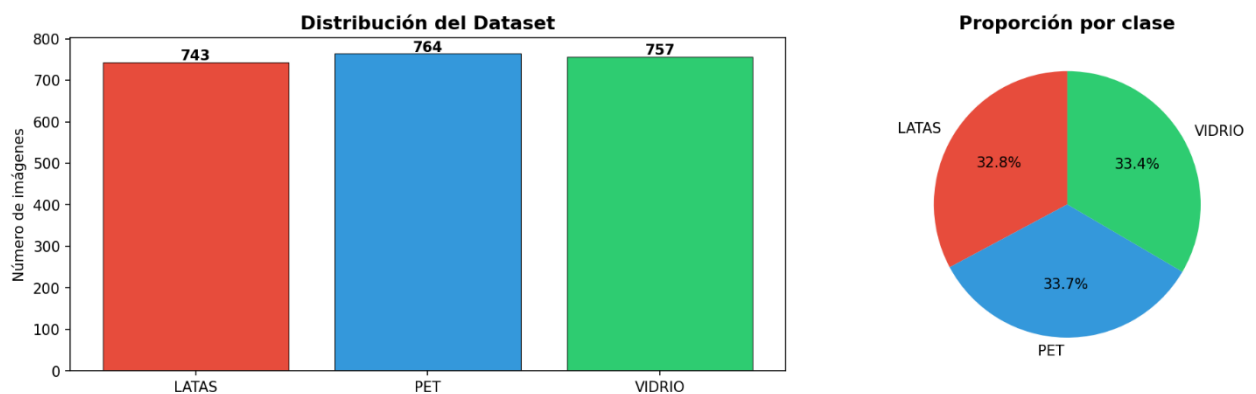
Métrica / Parámetro	Valor
Desbalance máximo entre clases	2,7 % → Clasificación: Balanceado (umbral < 10 %)
Resolución de entrada configurada	224 × 224 píxeles
Formatos de imagen aceptados	JPG, JPEG, PNG
Imágenes de entrenamiento (70 %)	1.584
Imágenes de validación (15 %)	339
Imágenes de test (15 %)	341
Semilla de reproducibilidad (seed)	42
Batch size	32
Num workers (DataLoader)	2
Normalización — media [R, G, B]	[0.485, 0.456, 0.406] (estadísticas ImageNet-1K)
Normalización — desv. típica [R, G, B]	[0.229, 0.224, 0.225] (estadísticas ImageNet-1K)

Nota. dataset_metadata.json; split_indices.json; pilar1_ingenieria_dato.ipynb, celdas 8, 10 y 20; config.json.

6.2 Visualizaciones Generadas

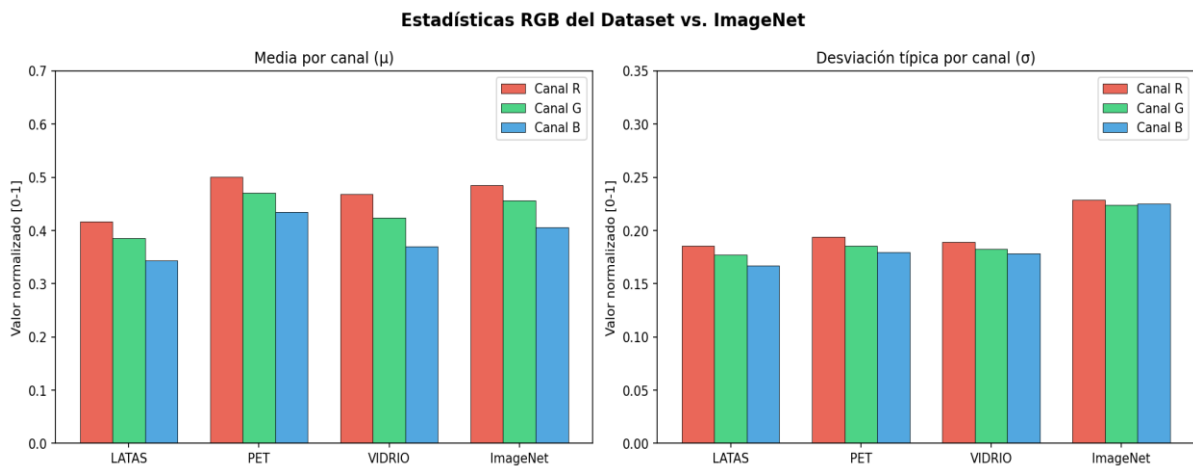
6.2.1 Distribución del Dataset (distribucion_dataset.png)

La figura muestra dos gráficas complementarias: un gráfico de barras con el número exacto de imágenes por clase (743, 764, 757, con etiquetas sobre cada barra) y un diagrama de sectores con los porcentajes correspondientes (32,8 %, 33,7 %, 33,4 %). La homogeneidad visual de ambas representaciones confirma el equilibrio entre clases y valida la decisión de no aplicar técnicas de rebalanceo. Esta figura constituye la evidencia visual del análisis de desbalance documentado en la sección 2.3.

Ilustración 1. Distribución del dataset de residuos reciclables por clase**6.2.2 Ejemplos del Dataset (ejemplos_dataset.png)**

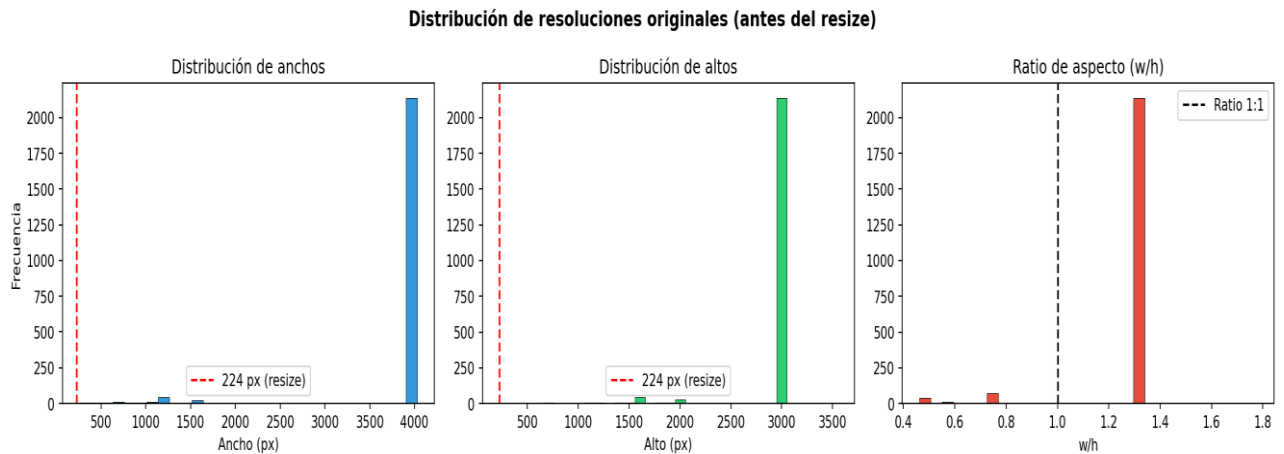
La figura muestra una cuadrícula de 3 filas $\times$ 5 columnas (una fila por clase) con 5 muestras aleatorias por clase. Esta visualización cumple una función de auditoría interna: permite verificar visualmente que el etiquetado es correcto, que la variabilidad de fondos, ángulos e iluminación es representativa, y que las imágenes no contienen artefactos.

Ilustración 2. Ejemplos del dataset por clase: cuadrícula 3 $\times$ 5 con 5 muestras por clase**6.2.3 Estadísticas RGB por canal y clase (estadisticas_rgb.png)**

Ilustración 3. Estadísticas RGB del Dataset vs ImageNet

La figura muestra las medias (μ) y desviaciones típicas (σ) de los valores de píxel por canal R, G, B para cada clase, comparadas con las estadísticas de ImageNet-1K. Respecto a las medias, las tres clases presentan valores entre 0,41 y 0,50, próximos a los de ImageNet (0,485/0,456/0,406), con la clase LATAS como la más oscura y PET como la más próxima a la referencia. En todos los casos el canal R domina sobre G y B, patrón consistente con ImageNet. Respecto a las desviaciones típicas, el dataset propio presenta valores de σ entre 0,17 y 0,19, inferiores a los de ImageNet (~0,23), lo que refleja la mayor homogeneidad del dominio específico frente a la diversidad de ImageNet-1K. Esta diferencia en σ es esperable dado que el dataset cubre únicamente tres tipos de envases en exteriores, frente a las 1.000 clases heterogéneas de ImageNet. En conjunto, la proximidad de las medias y la dirección coherente de las desviaciones validan el uso de las estadísticas de ImageNet como referencia de normalización sin necesidad de recalcularlas sobre el dataset propio.

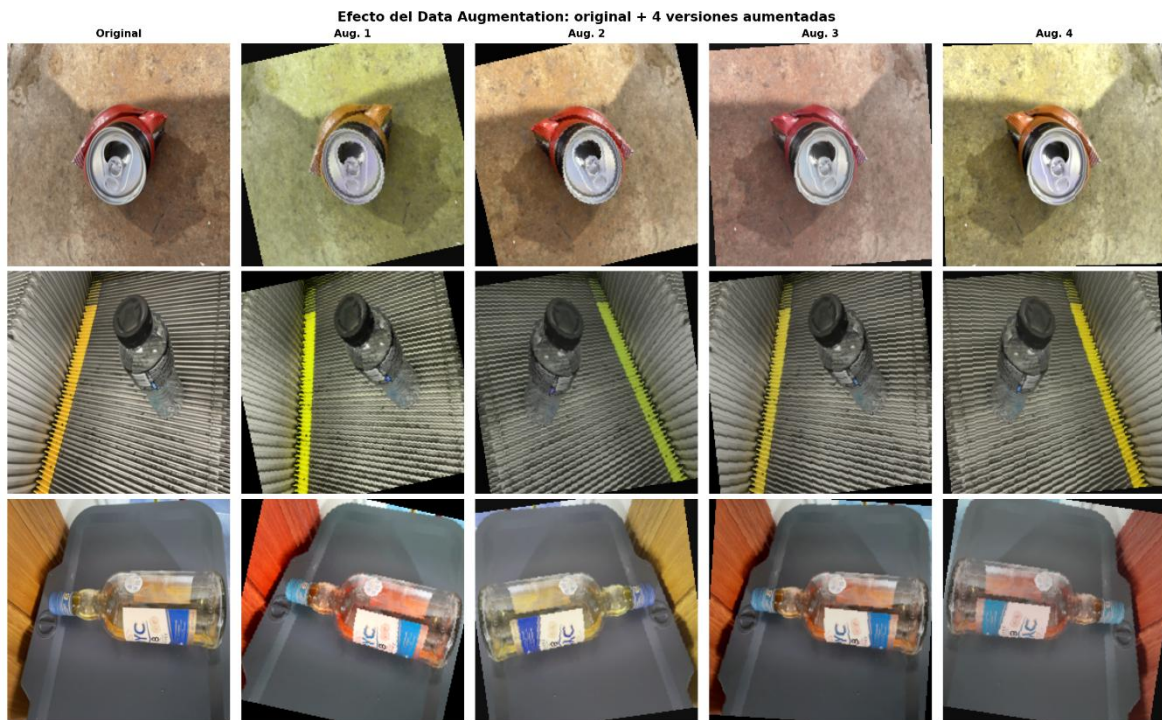
6.2.4 Distribución de resoluciones originales (resoluciones_originales.png)

Ilustración 4.Distribución de resoluciones originales

La figura muestra que el dataset presenta una distribución de resoluciones altamente homogénea: prácticamente la totalidad de las imágenes tienen una resolución nativa de aproximadamente 4.032×3.547 píxeles (ancho medio 3.867 px, alto medio 2.946 px), capturadas con el mismo dispositivo móvil bajo un ratio de aspecto medio de 1,30 (próximo al estándar 4:3). Un número muy reducido de imágenes presenta resoluciones inferiores, con un mínimo de 259×561 px, correspondientes a capturas con configuración diferente. Esta homogeneidad confirma que no existe variabilidad significativa en el hardware de captura y que el redimensionado a 224×224 px implica una reducción de escala de aproximadamente 17:1 en el ancho. A pesar de esta reducción, la resolución resultante es suficiente para preservar los patrones discriminativos de los tres materiales, como confirman los resultados del Pilar 2 (F1-Score > 98 %).

6.2.5 Efecto del data augmentation (augmentation_efecto.png)

Ilustración 5. Efecto del Data Augmentation



La figura muestra, para una imagen representativa de cada clase, la versión original redimensionada junto a cuatro versiones aumentadas generadas por `transform_train`. El efecto de cada técnica es visible en las tres clases: las rotaciones de hasta $\pm 15^\circ$ producen inclinaciones apreciables del envase respecto al eje vertical (especialmente en LATAS y PET); el volteo horizontal invierte la orientación del envase de forma semánticamente válida (claramente visible en VIDRIO); y ColorJitter introduce variaciones de brillo, contraste y saturación que modifican notablemente la apariencia cromática de los envases, simulando distintas condiciones de iluminación (en LATAS los tonos pasan de cálidos a verdosos; en VIDRIO la botella oscila entre amarillo neutro y naranja intenso). Las esquinas negras en algunas versiones aumentadas son el artefacto geométrico esperado de la rotación sobre fondo negro, que el modelo aprende a ignorar durante el entrenamiento. RandomErasing no se visualiza directamente porque opera sobre tensores normalizados; su efecto consiste en sustituir regiones rectangulares aleatorias de entre el 2 % y el 15 % del área con valores de tensor fijos, simulando oclusiones parciales del envase.

7. Conclusiones y Consideraciones para Producción

7.1 Síntesis del Pilar 1

El Pilar de Ingeniería del Dato ha completado satisfactoriamente todos sus objetivos, generando los siguientes entregables verificados:

- Un dataset original de 2.264 imágenes propias de tres clases de envases reciclables, capturadas en condiciones deliberadamente heterogéneas para maximizar la robustez del modelo final frente a la variabilidad operativa real de un sistema SDDR.
- Un análisis exploratorio que confirma un desbalance máximo del 2,7 % entre clases, nivel que no requiere técnicas de mitigación activa y permite el uso de Cross-Entropy estándar y F1-Score ponderado como métricas principales.
- Un pipeline de preprocesamiento completamente justificado bibliográficamente: resolución 224×224 px (He et al., 2016; Tan y Le, 2019; Liu et al., 2022), normalización con estadísticas ImageNet (Malik et al., 2022), y cuatro técnicas de augmentation on-the-fly diseñadas para simular condiciones SDDR reales (Zeng, 2024; Mahmood y Minallah, 2023).
- Una partición reproducible 70/15/15 con semilla fija (seed = 42), cuyos índices están persistidos en split_indices.json, garantizando la trazabilidad con el Pilar 2 y la ausencia de data leakage.
- Tres DataLoaders correctamente configurados con transformaciones independientes por partición, previniendo cualquier forma de contaminación del conjunto de evaluación.

7.2 Consideraciones para el Deployment en Producción (SDDR)

De cara al deployment del sistema en máquinas SDDR reales, el pipeline del Pilar 1 presenta las siguientes consideraciones de producción:

Tabla 11. Consideraciones para el deployment del pipeline en producción (SDDR)

. Consideraciones para el deployment del pipeline en producción (SDDR)

Aspecto	Implementación en el TFG				Consideración para Producción
Resolución entrada	de	224	×	224 px	Cámara RGB HD con resolución nativa $\geq 1080p$; redimensionado a 224×224 px antes de inferencia mediante OpenCV o PIL
		(redimensionado software)		por	

Aspecto	Implementación en el TFG	Consideración para Producción
Normalización	Estadísticas fijas [0.485, 0.456, 0.406]	Mantener las mismas estadísticas para consistencia con los pesos preentrenados
Augmentation	On-the-fly exclusivamente durante entrenamiento	No aplicar augmentation en inferencia: solo transform_val_test (resize + normalize)
Actualización del dataset	Dataset estático v1.0 (2.264 imágenes)	Protocolo de reentrenamiento periódico con nuevas imágenes capturadas en campo (active learning)
Cobertura de clases	3 clases: LATAS, PET, VIDRIO	Posible extensión a tetra brik u otros materiales según evolución del marco regulatorio (PPWR 2029)
Hardware de cámara	Imágenes capturadas con dispositivo móvil	Cámara industrial fija integrada en la máquina SDDR; posición y distancia constantes reducen la variabilidad

7.3 Limitaciones Identificadas

- Tamaño del dataset: 2.264 imágenes es suficiente para transfer learning con los resultados obtenidos en el Pilar 2 (F1-Score > 98 %), pero inferior a los datasets industriales. La generalización a entornos operativos muy distintos de los de captura podría requerir ampliación mediante recolección de campo.
- Variabilidad no capturada: el protocolo de captura no incluye variaciones estacionales, condiciones de lluvia o polvo en exteriores, ni suciedad extrema de los envases. Para deployment en exteriores, sería necesario ampliar el dataset con estas condiciones.
- Sesgo de captura: todas las imágenes fueron capturadas por la misma persona con el mismo dispositivo, lo que puede introducir un sesgo sistemático en distancia y ángulo. En producción, la cámara del SDDR tendrá posición fija, lo que en realidad reduce esta variabilidad y mejora la consistencia.

- Etiquetado manual: todas las imágenes han sido etiquetadas manualmente por el autor. Dado el número reducido de clases (3) y la alta distinguibilidad visual entre materiales, el riesgo de error de etiquetado es bajo; no obstante, un protocolo de validación cruzada del etiquetado sería recomendable en una versión de producción.

8.Referencias Bibliográficas

Las referencias se presentan en orden alfabético por apellido del primer autor, con formato APA 7ª edición (American Psychological Association, 2020). Los títulos de revistas científicas y libros aparecen en cursiva; los títulos de artículos y ponencias de congreso se presentan en redonda. Los identificadores DOI enlazan a las versiones publicadas de los trabajos.

- Du, J., Xu, L., Wang, H., & Dong, W. (2024). Zero-shot day-night domain adaptation with a physics prior. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2023)* (pp. 21533–21543). IEEE. <https://doi.org/10.1109/ICCV51070.2023.01970>
- Gobierno de España. (2022). *Ley 7/2022, de 8 de abril, de residuos y suelos contaminados para una economía circular*. Boletín Oficial del Estado, núm. 85, de 9 de abril de 2022. <https://www.boe.es/eli/es/l/2022/04/08/7/con>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016)* (pp. 770–778). IEEE. <https://doi.org/10.1109/CVPR.2016.90>
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022)* (pp. 11976–11986). IEEE. <https://doi.org/10.1109/CVPR52688.2022.01167>
- Mahmood, A., & Minallah, N. (2023). A comprehensive survey on data augmentation techniques for classification and segmentation. *IEEE Access*, 11, 89887–89911. <https://doi.org/10.1109/ACCESS.2023.3306345>
- Malik, M., Sharma, S., Uddin, M., Chen, C.-L., Wu, C.-M., & Soni, P. (2022). Waste classification for sustainable development using image recognition with deep learning neural network models. *Sustainability*, 14(12), Artículo 7222. <https://doi.org/10.3390/su14127222>
- Olorunnisola, F. E., Adegun, A. A., Mirkes, E. M., Adetiba, E., & Ekong, V. E. (2025). A novel three-stage approach for efficient waste classification: Integrating depthwise parallel convolutional neural network and ensemble extreme learning machine.

Expert Systems with Applications, 262, Artículo 125557.
<https://doi.org/10.1016/j.eswa.2024.125557>

Parlamento Europeo y Consejo de la Unión Europea. (2024). *Reglamento (UE) 2025/40 del Parlamento Europeo y del Consejo, de 19 de noviembre de 2024, sobre envases y residuos de envases*. Diario Oficial de la Unión Europea, L 2025/40, de 22 de enero de 2025. https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=OJ:L_202500040

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML 2019), Proceedings of Machine Learning Research*, 97, 6105–6114. PMLR. <https://proceedings.mlr.press/v97/tan19a.html>

Zeng, H. (2024). Data augmentation methods in deep learning for image classification: A comprehensive survey. *Journal of Imaging Science and Technology*, 68(2), Artículo 020501. <https://doi.org/10.2352/J.ImagingSci.Technol.2024.68.2.020501>

Zhang, Y., Wei, X.-S., Zhou, J., & Wu, J. (2023). Source-free domain adaptation guided by vision and vision-language pre-training. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(4), 2164–2178. <https://doi.org/10.1109/TPAMI.2023.3322747>